

Animations and States

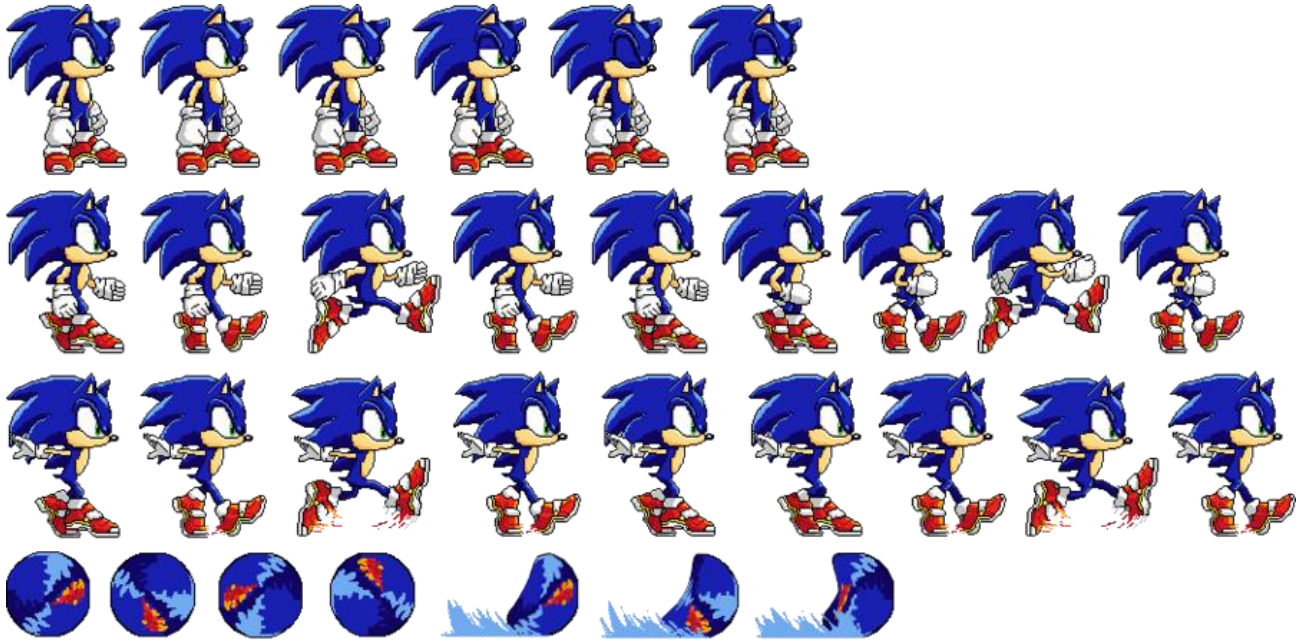
Animation

- Animation in a 2D Unity game can be done in one of two methods: 1)using a character **sprite sheet**, or using a 2)character **atlas**.
- You are free to choose either for your project, however the lab session will focus on animation using a character **sprite sheet**.
- For this method to be successful, you must first have your character drawn in all the positions and movements it needs. You can draw in any software you find comfortable (e.g. Photoshop, Affinity Designer, Aseprite, PixieEditor 2.0, etc.)
- Second, save them all in the same image file in PNG format
- Third, import the sprite sheet into Unity, set the Sprite Mode to Multiple, and open the Sprite Editor

To Animate the Player

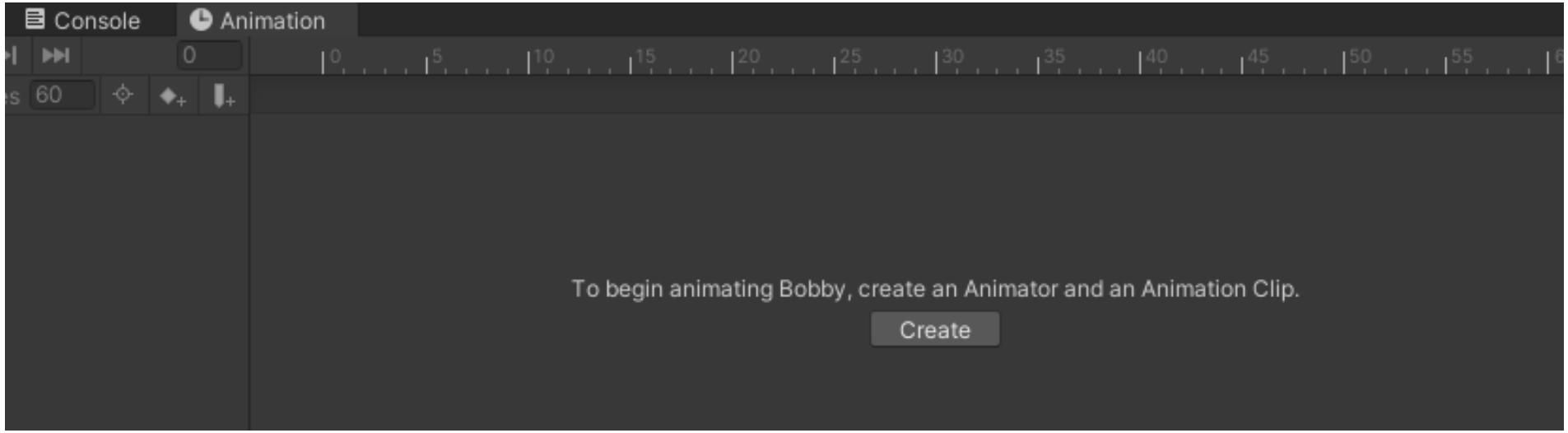
- Choose (or create) a sprite sheet containing all the character's poses and movements, like the examples below:





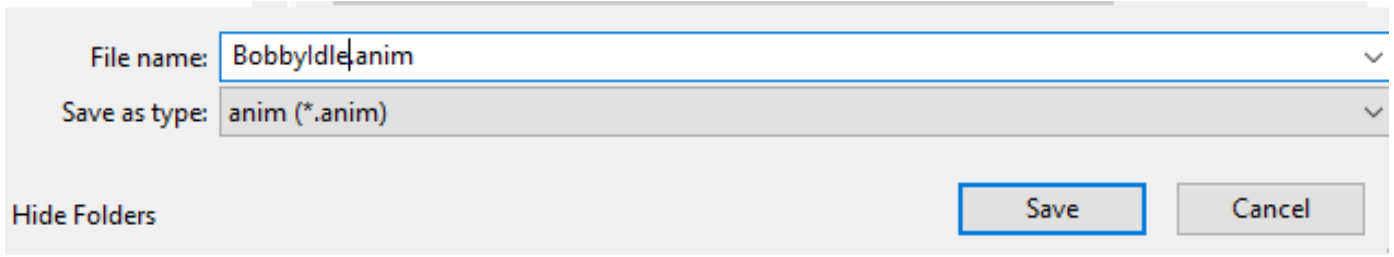
Exercise #07 – Create Animations

- For best results, the drawings on your sprite sheet should all be the same size (same width x height). Slice your sprite sheet (see Lab 2)
- Go to Window in the Taskbar above, and open the Animation window and dock it somewhere.
- Make sure your Player character **is selected in the Hierachy**, go to the Animation window and click Create.

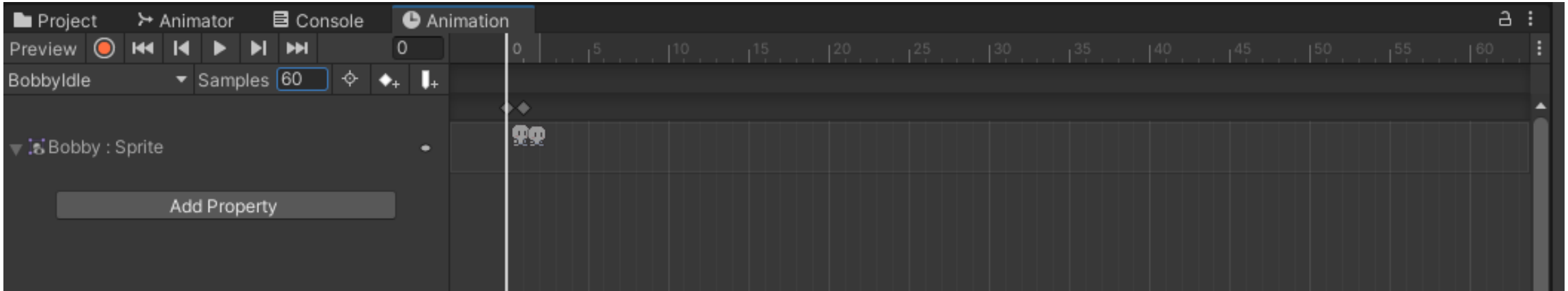


Exercise #07 – Create Animations (Cont.)

- Name your animation
- You need to create an animation file for every type of movement, so for example if you have art for the character's idle stance, walk and jump cycles, you're going to create 3 animations in total.

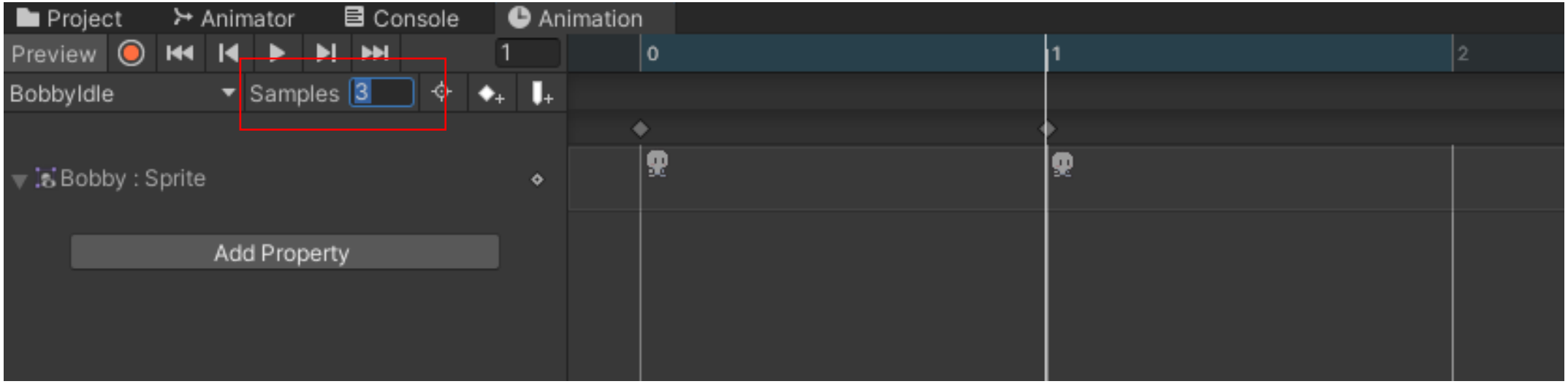


- To create the idle animation, drag and drop the idle sprites into the Animation window like this



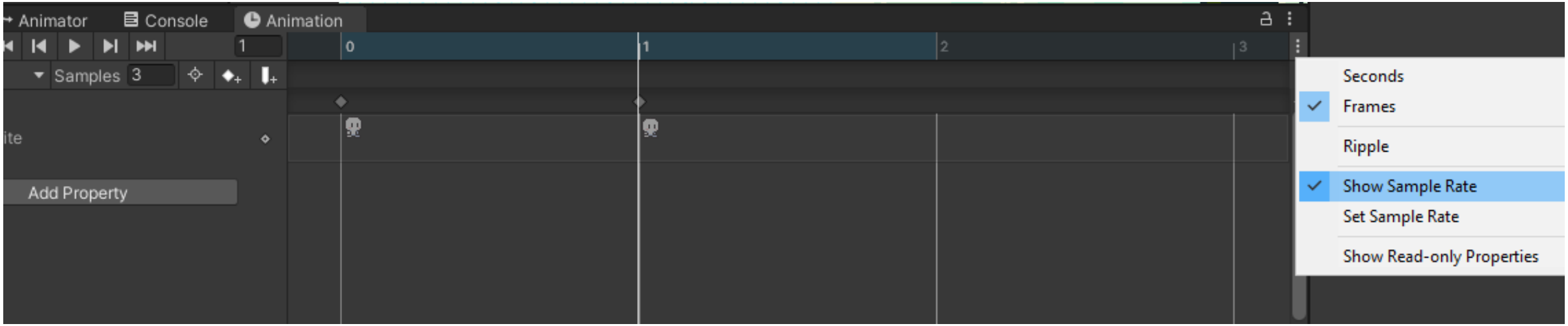
Exercise #07 – Create Animations (Cont.)

- Change the value for ‘Samples’ to whatever value is suitable to avoid having the animation play too fast or too slow. To test your animation’s fluidity, click the Play button above the Scene window



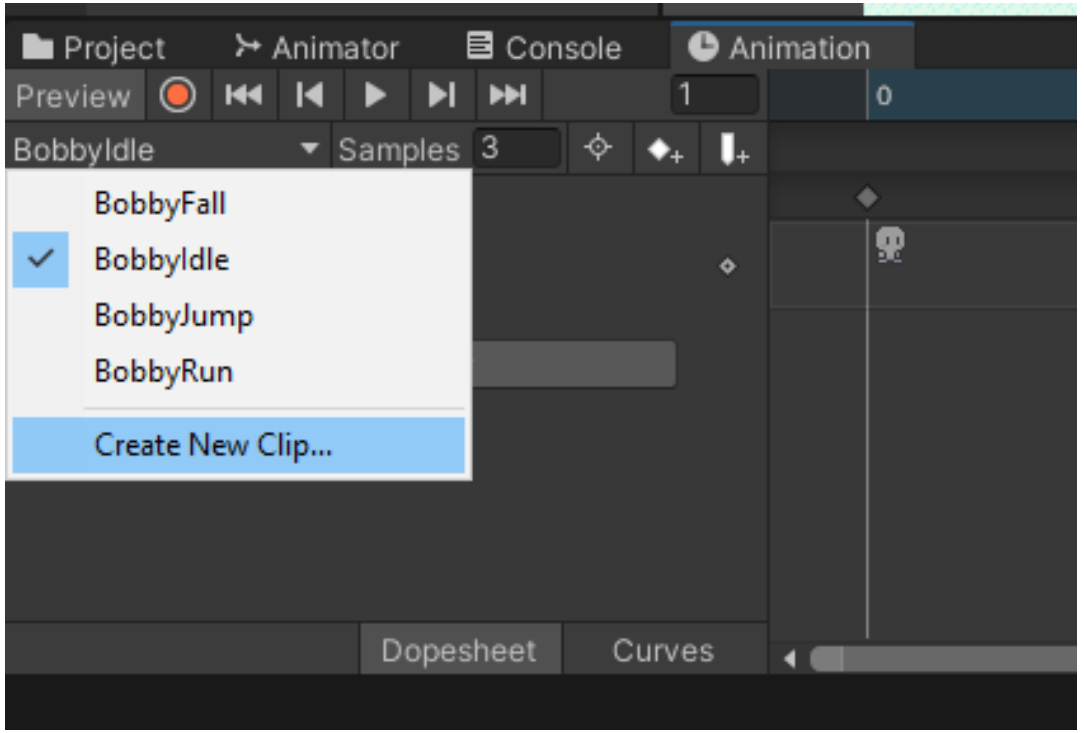
Exercise #07 – Create Animations (Cont.)

- Quick note:
- If you work with a version that doesn't automatically show the 'Samples' property, click on the 3 vertical dots on the right side of the Timeline, and enable 'Show Sample Rate'



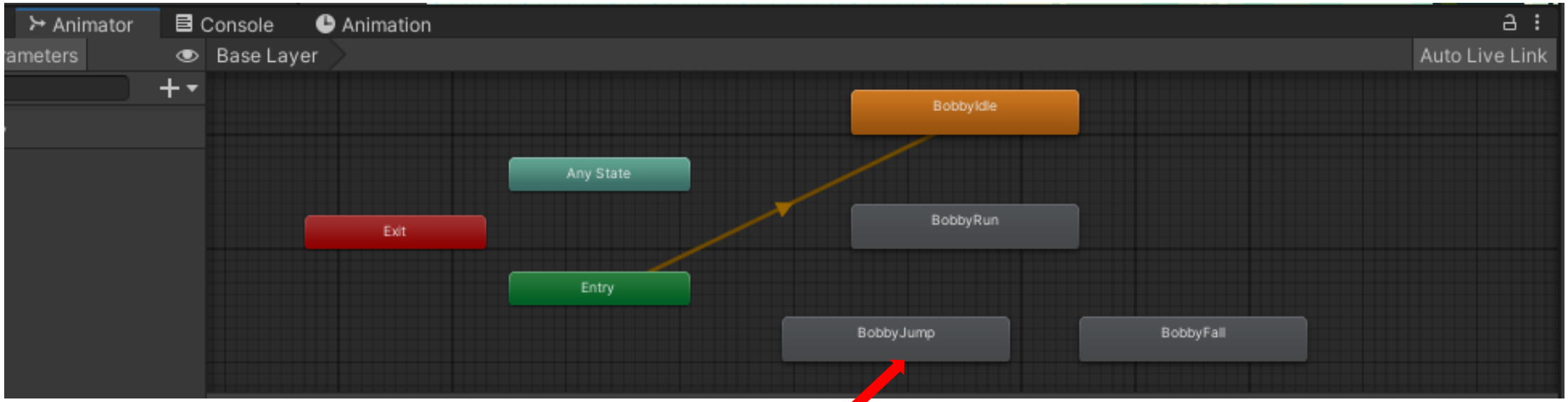
Exercise #07 – Create Animations (Cont.)

- When you're satisfied with the animation, save it, then create the animations for run, jump and fall
  - To create a new animation clip, click the bar underneath the Record button



Exercise #07 – Create Animations (Cont.)

- Once you've finished all animations, go to the taskbar above and choose >Window>Animator. You'll find something like this:



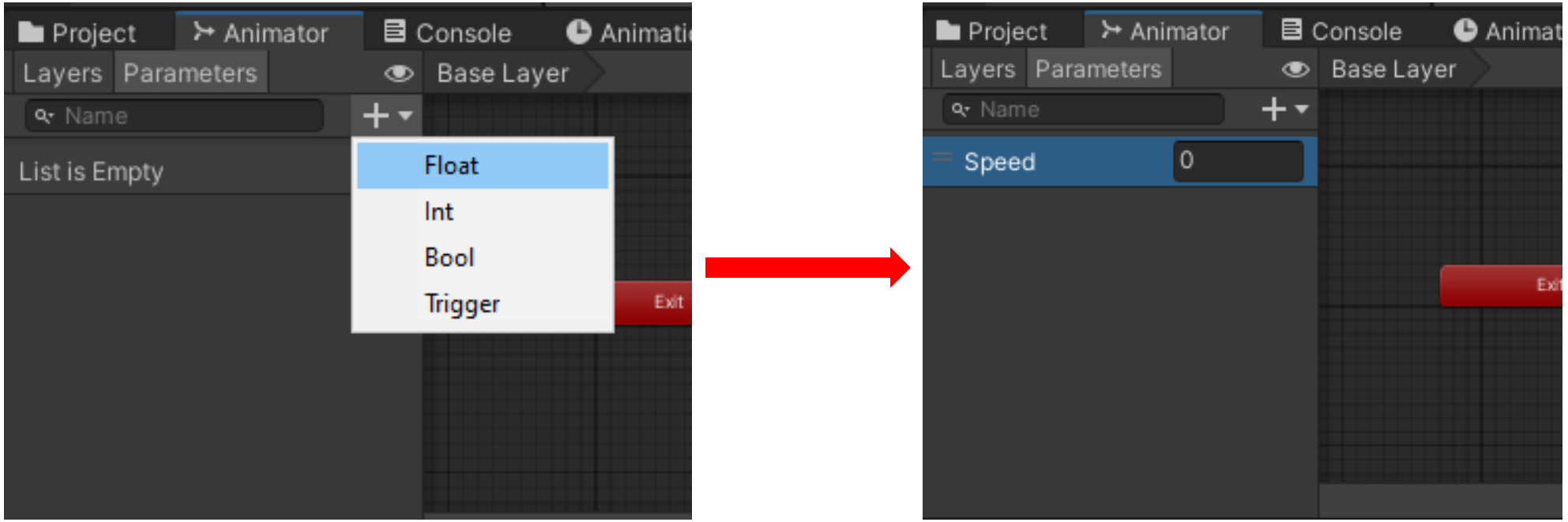
Quick tip:
If you don't want the jump animation to keep looping, double click on the Jump state and uncheck Loop Time in the Inspector.

Exercise #08 – Set Animation Conditions

- Now you must begin linking your animations with your code
- Go to your player controller script
- Add an Animator type to your list of variables
  - **private Animator anim;** //instance of Animator object to control the character's animation in code
- Go into your Start() function and add
  - **anim = GetComponent<Animator>();**//The player now has an Animator component attached to it, and the animation will play accordingly when the character moves

Exercise #08 – Set Animation Conditions (Cont.)

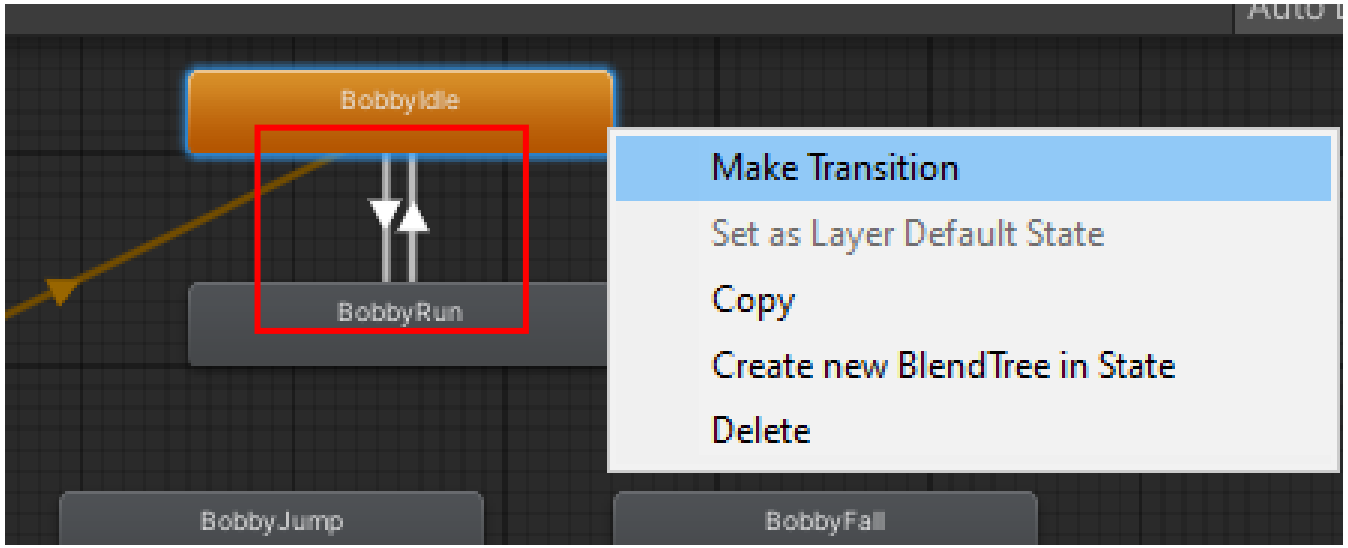
- Now go back to the Animator tab in Unity
- You'll find a tab called Parameters on the left side. Click the + sign to create a new Float and name it Speed



Exercise #08 – Set Animation Conditions (Cont.)

- The orange state represents the first animation your character will play when the level begins. Normally, an idle animation is the default

- To allow transition between states (e.g. Idle to Walk), right click on the idle state and choose Make Transition. **Drag** the arrow to the run state then do the same back from run state to the idle state.

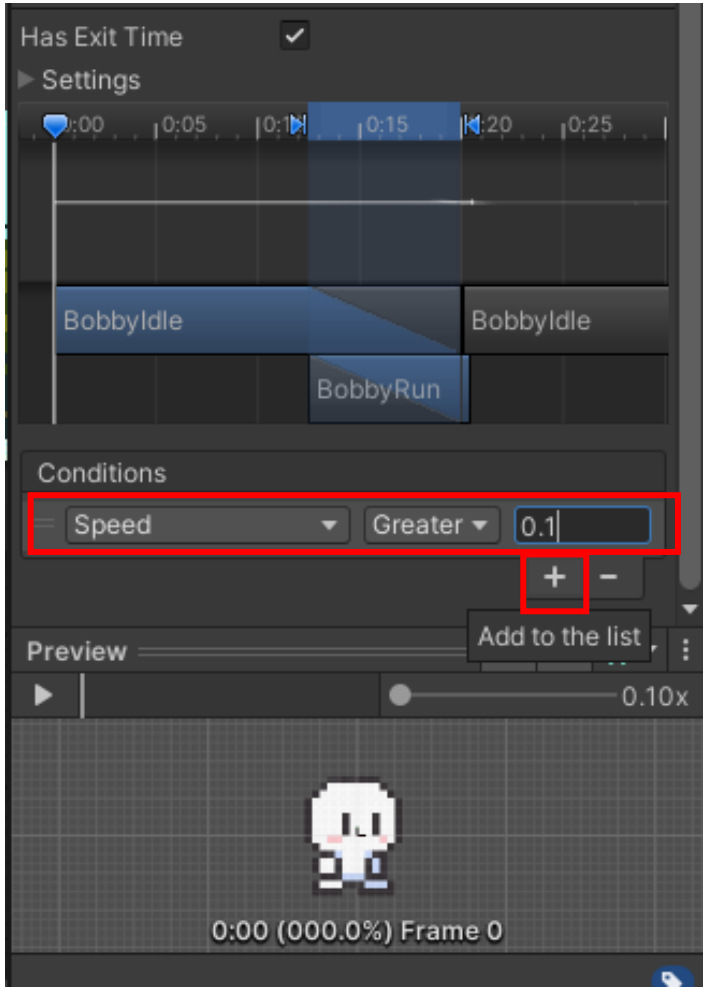


Exercise #08 – Set Animation Conditions (Cont.)

- Click on the arrow from the idle to run animation, and go to the Inspector, and click the + sign under Conditions.

It will create a Speed condition for you. Adjust the Condition so that the Speed from idle to running is Greater than 0.1.

- Now click on the arrow going from run to idle, but make the Speed's condition Less than 0.1.



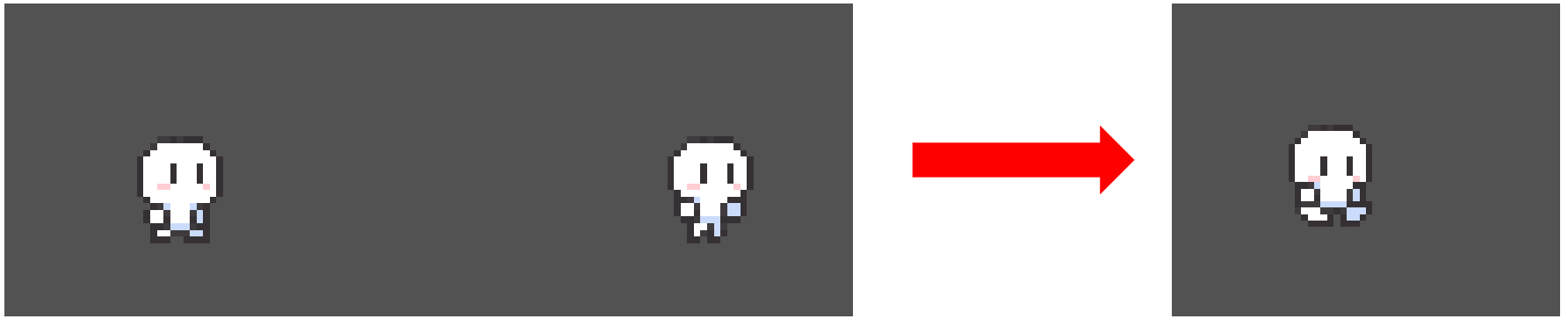
Exercise #08 – Set Animation Conditions (Cont.)

- Go back to your code, and write a statement that connects the character's velocity on the x-axis with the Speed condition we have added in the parameters of the Animator

- Underneath your basic left-right movement code, add:
  - anim.SetFloat("Speed",Mathf.Abs(GetComponent<Rigidbody2D>().velocity.x));
  - this function takes the Animator's parameter name(in this case Speed), and gives it the value of the character's current velocity on the x-axis. However, the fact that x could be a negative value can cause problems, so Mathf.Abs turns any -ve number into positive without affecting the player character's direction

Exercise #09 – Blend Trees

- When it comes to jumping, we'll use another Float parameter in the animator so that when the y-axis value changes +vely, the character's jumping animation is activated, and when the value changes –vely, the fall animation is activated.



Jump up (+ve y)

Fall down (-ve y)

Exercise #09 – Blend Trees (Cont.)

What exactly is a Blend Tree?

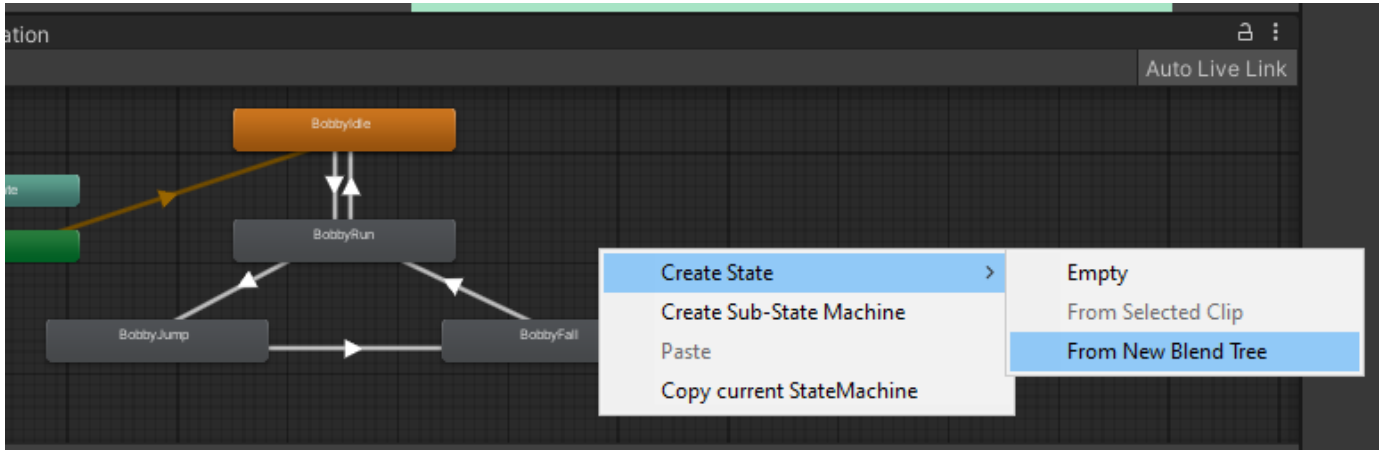
- It's a tool within the Animation State Machine that allows you to create fluid transitions between two or more similar motions (e.g. blending between walking and running animations depending to the character's speed)

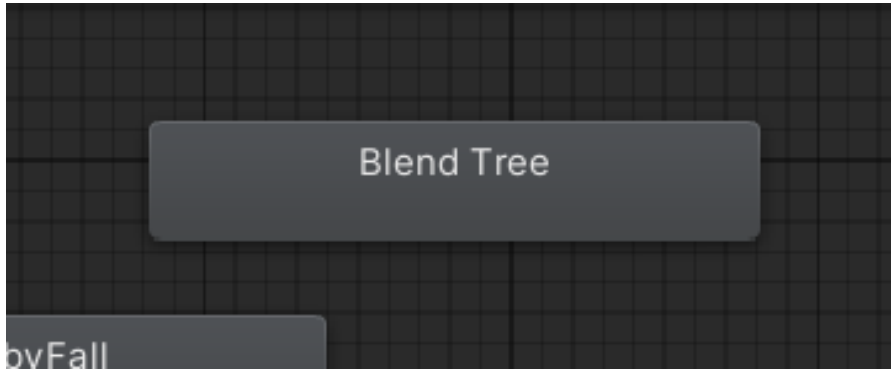
- It's also useful when trying to blend in animations that cannot be directly controlled by the player (e.g. a player can technically initiate a walking animation directly by making the character walk, but he can't directly initiate a falling animation.)

- See Unity's own page explaining Blend Trees **here**.

Exercise #09 – Blend Trees (Cont.)

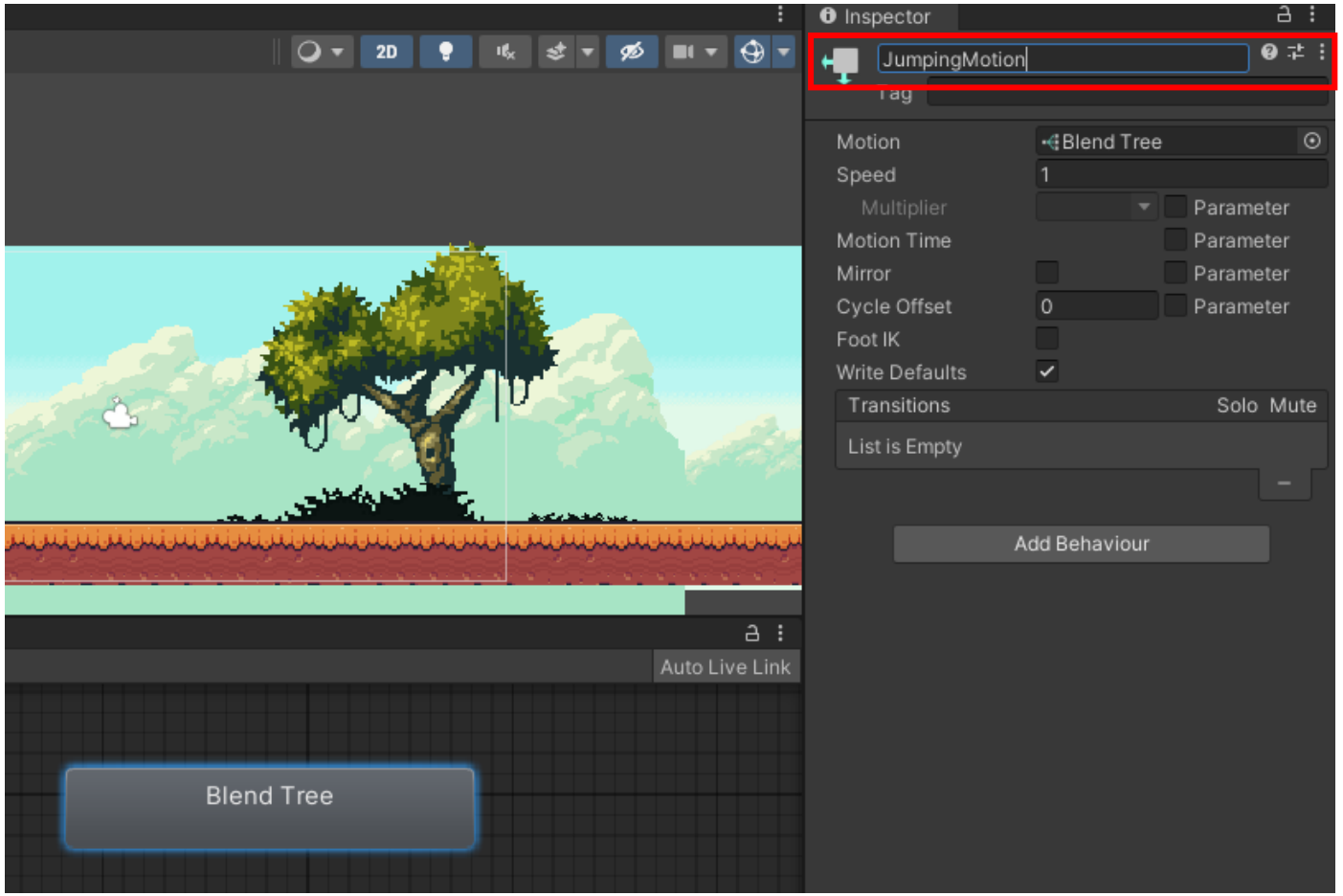
- Right click in an empty space and create a new Blend Tree.





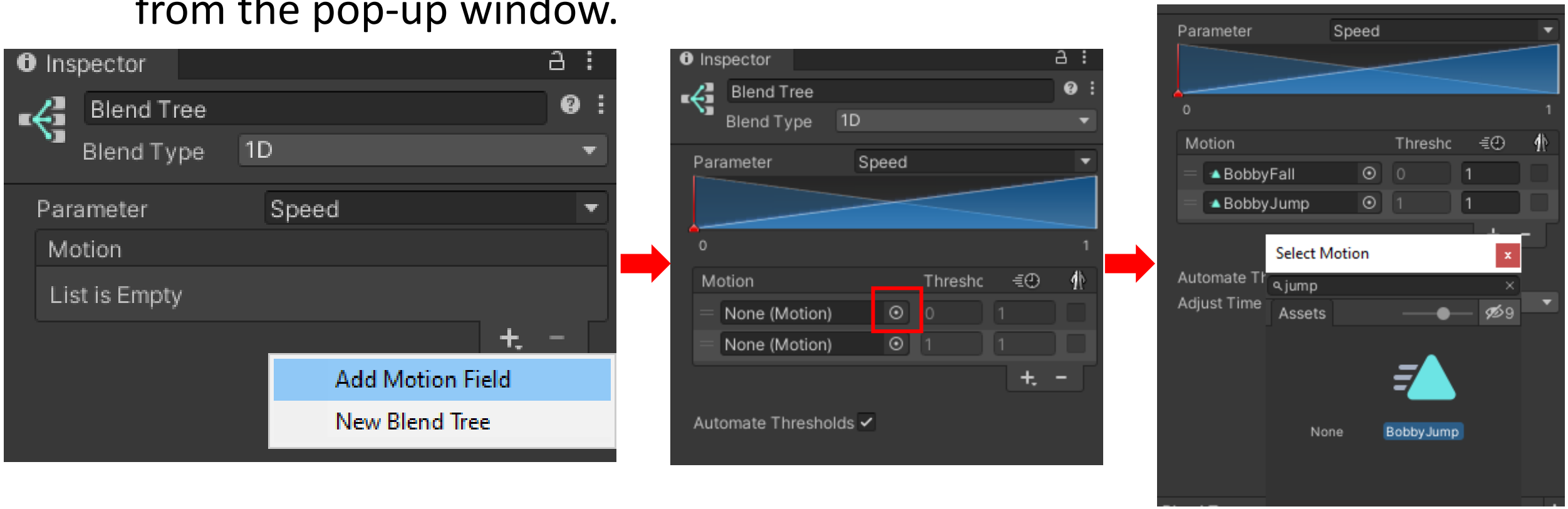
Exercise #09 – Blend Trees (Cont.)

- Name it Jumping Motion



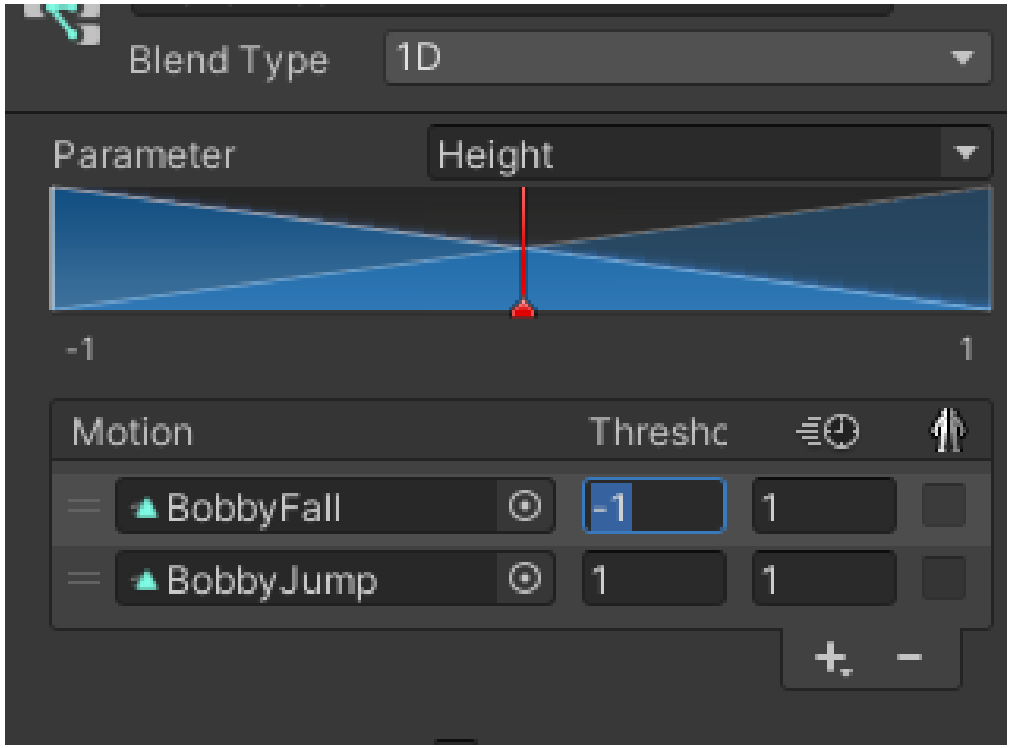
Exercise #09 – Blend Trees (Cont.)

- Double click the tree again and add 2 motion fields, one containing the fall animation and the other containing the jump. Select them from the pop-up window.



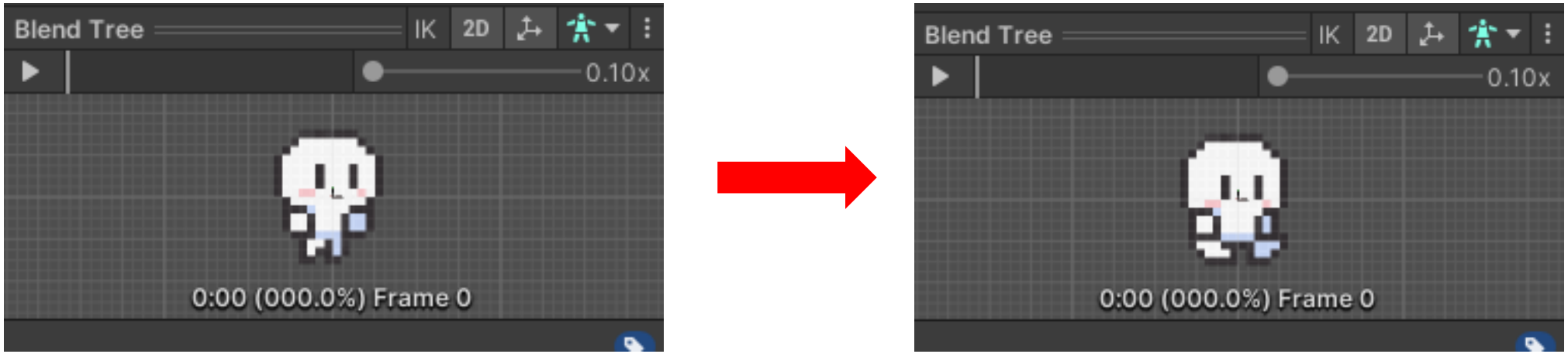
Exercise #09 – Blend Trees (Cont.)

- Don’t forget to **uncheck Automate Thresholds**. We’ll insert our own values.
- Make the threshold value for Fall = -1. Make the value for Jump 1.



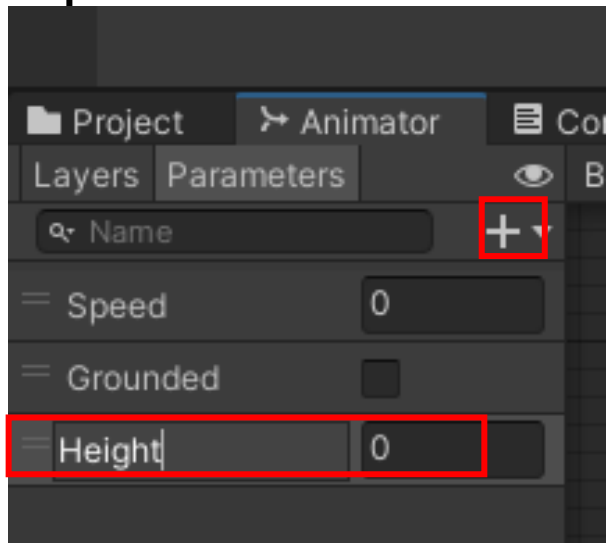
Exercise #09 – Blend Trees (Cont.)

- This makes the change between jump and fall animations very smooth. Whenever the velocity on the y-axis moves +vely, animation for the jump plays, and whenever the velocity drops towards 0 and beyond, value becomes –ve and falling animation plays.

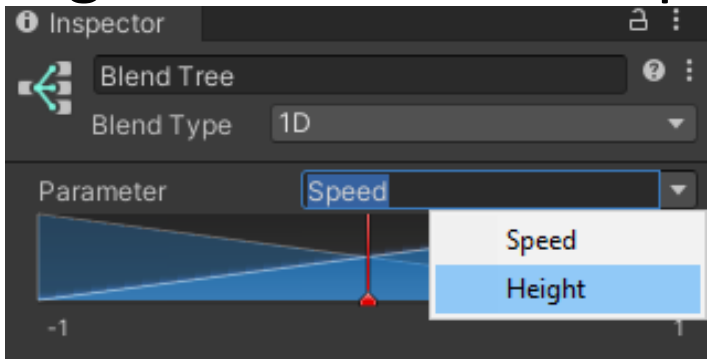


Exercise #09 – Blend Trees (Cont.)

- Now create another Float parameter for the Animator and name it Height.

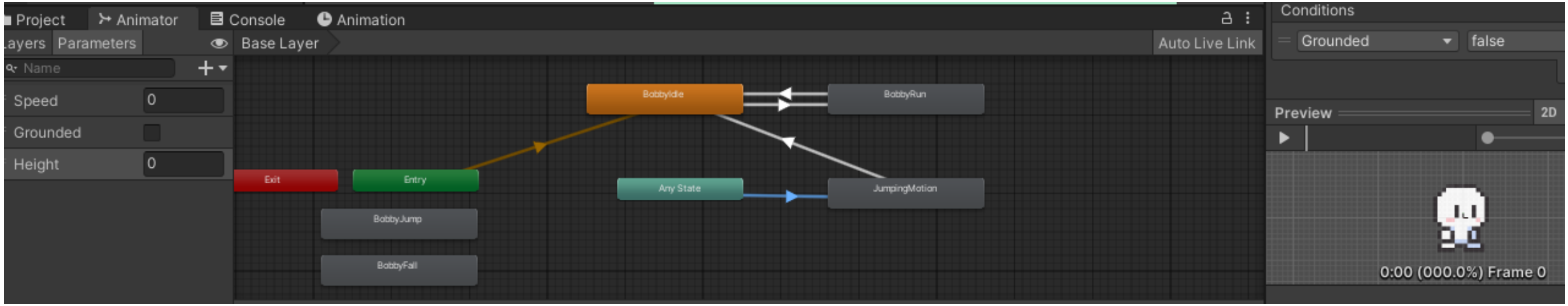


- Don’t forget to also change the Blend Tree’s parameter to Height.



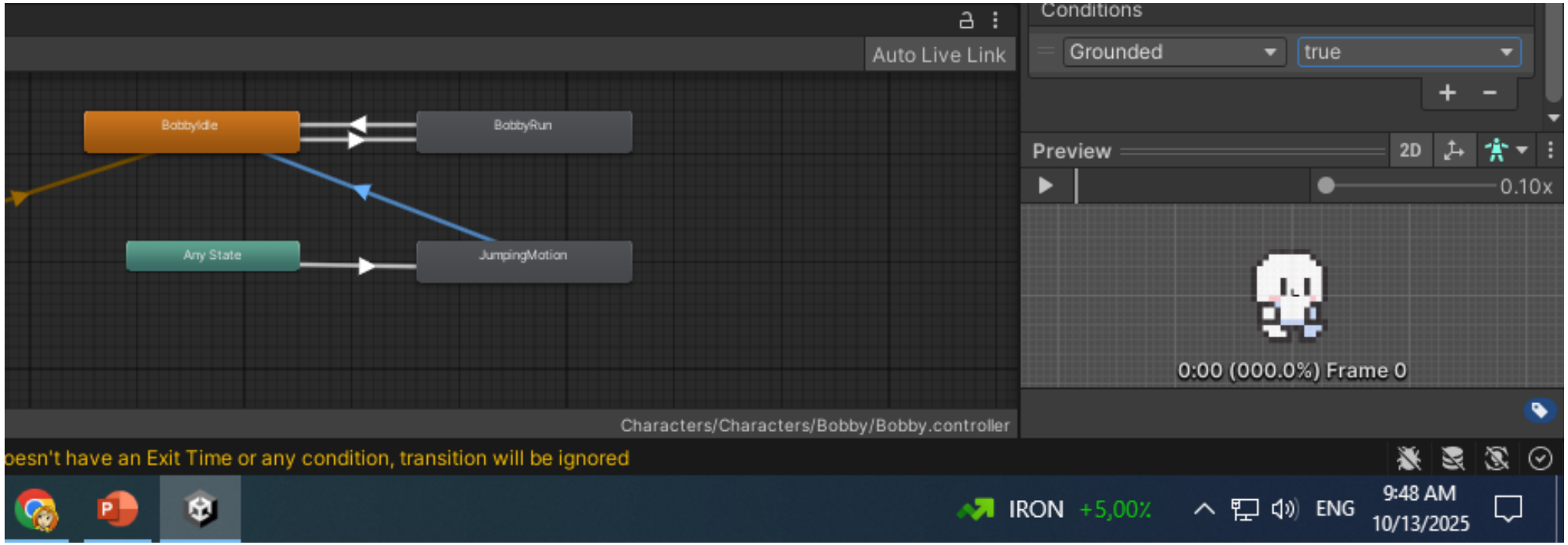
Exercise #09 – Blend Trees (Cont.)

- Now go back to the Base Layer in the Animator and create a transition arrow from Any State to the Jumping Blend Tree.
- Create a new Bool parameter in the Animator, name it Grounded.
- Make sure the condition on the arrow is false.



Exercise #09 – Blend Trees (Cont.)

- Then create another transition arrow from Jumping back to Idle, and make sure the Grounded condition is true.



Exercise #09 – Blend Trees (Cont.)

- Finally, go back to your script and add the following:

anim.SetFloat("Height", GetComponent<Rigidbody2D>().velocity.y);

This function takes the Animator's parameter name (Height), and gives it the value of the player's current velocity on the y-axis. Since we're using a Blend Tree, we *want* to keep track of whether y is +ve or -ve. +ve means the jump up animation will play, -ve means the fall down animation will play.

anim.SetBool("Grounded", grounded);

Animator understands that its parameter Grounded and the boolean value grounded are related.

Important Note:

- On each transition arrow, don’t forget to **uncheck** the ‘has exit time’ condition.
- Open the Settings menu, and also uncheck Fixed Duration, and make Transition Duration be zero.
- This removes delays when transitioning from one animation and the other.

